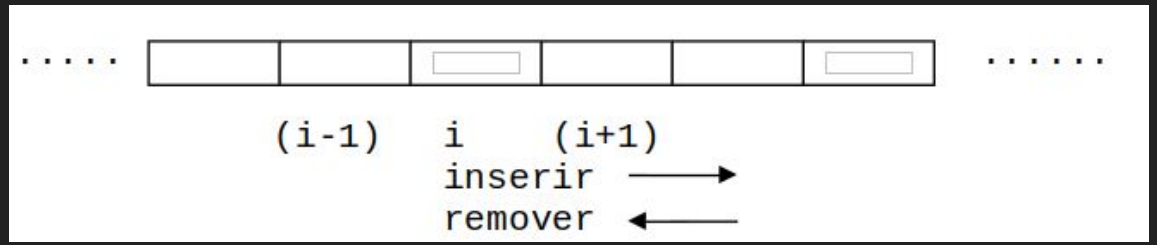
Alocação Dinâmica - Listas Encadeadas

Estruturas de Dados e Introdução à Projeto e Análise de Algoritmos

Prof. Luis Souza

Lista Sequencial

- Conjunto de elementos contíguos na memória: lista em Python é o melhor exemplo de lista “sequencial” (sets e dicts também).
  - O elemento i é precedido pelo elemento i-1 e seguido pelo elemento i+1.
  - Elementos contíguos: acesso por meio do seu índice - indica o deslocamento a partir do início.
- Vantagem: é possível o acesso direto (O(1)) a qualquer dos elementos da lista, mediante o conhecimento do índice.
- Desvantagem: inserção e remoção de novos elementos:
  - exige realocação dinâmica de memória ou deslocamento de elementos dependendo do sistema usado.

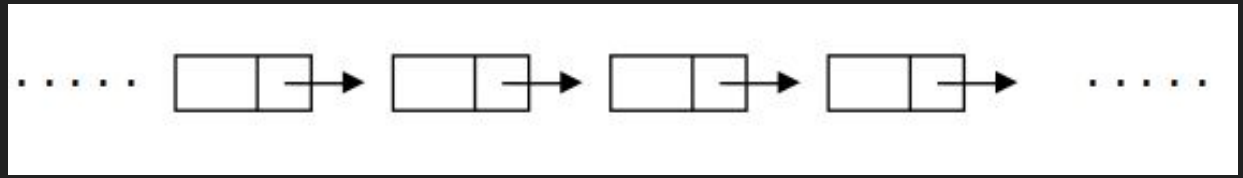


Lista Sequencial

- Operações de alocação dinâmica de memória e deslocamento de dados para a inserção e remoção de elementos não são eficientes.
- No Python, as operações append(), del(), pop(), extend(), etc., bem como as operações com sub-listas, embora bastante úteis e simples, não são operações eficientes: necessidade de deslocamento dos elementos ou uso adicional de memória.

Lista Encadeada (ou Ligada)

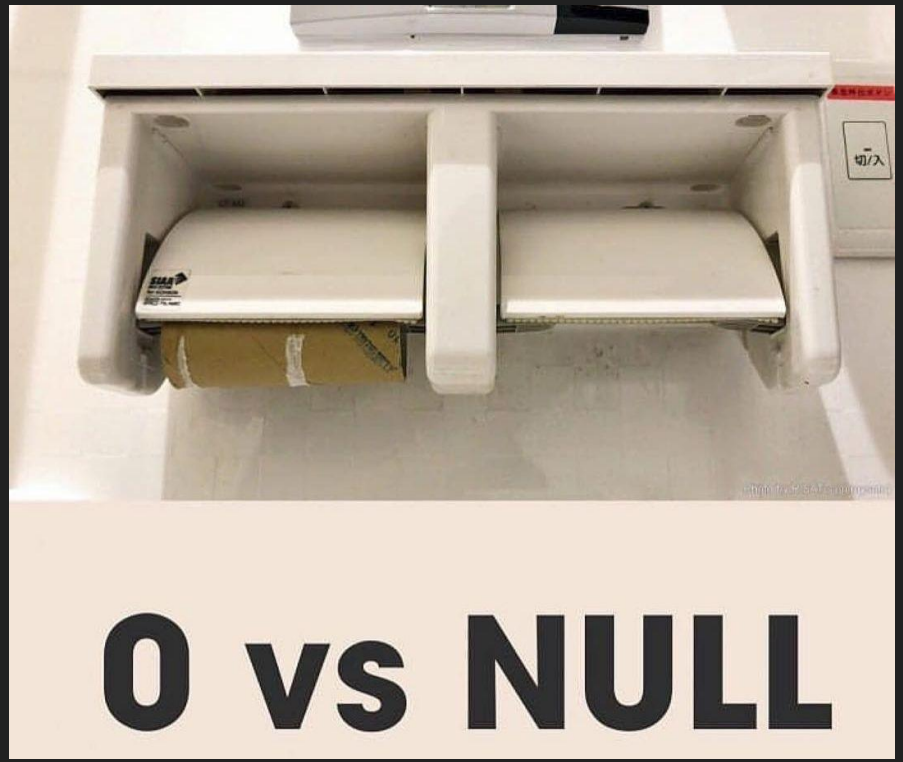
- Conjunto de elementos em que cada elemento indica qual o próximo - não existe contiguidade física entre os elementos - elementos vizinhos podem estar em posições físicas separadas na memória.
- Vantagem: flexibilidade na inserção e remoção de elementos.
- Desvantagem: não tem-se a opção de acesso direto aos elementos - não existe indexação para o acesso direto - é necessário percorrer a lista.



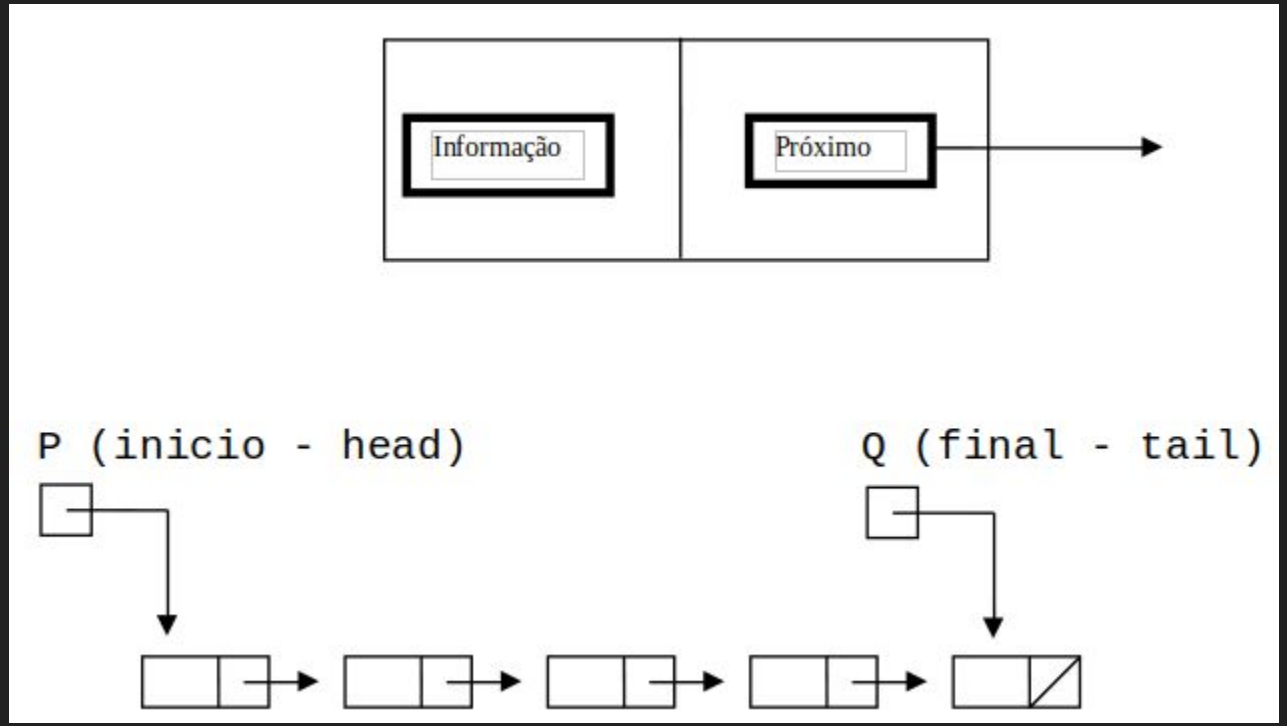
Lista Encadeada (ou Ligada)

- Cada item ou elemento da lista encadeada contém uma informação e **um ponteiro para o próximo item.**
- A lista em si é representada por um ponteiro para o seu início.
  - O próximo do último elemento é representado por “None” em Python (lembram-se do NULL?) para indicar que é o fim da lista ligada.
  - Um ponteiro para o último elemento pode ser útil em alguns casos (facilita operações no final da lista como inserção e remoção).

Lista Encadeada (ou Ligada)

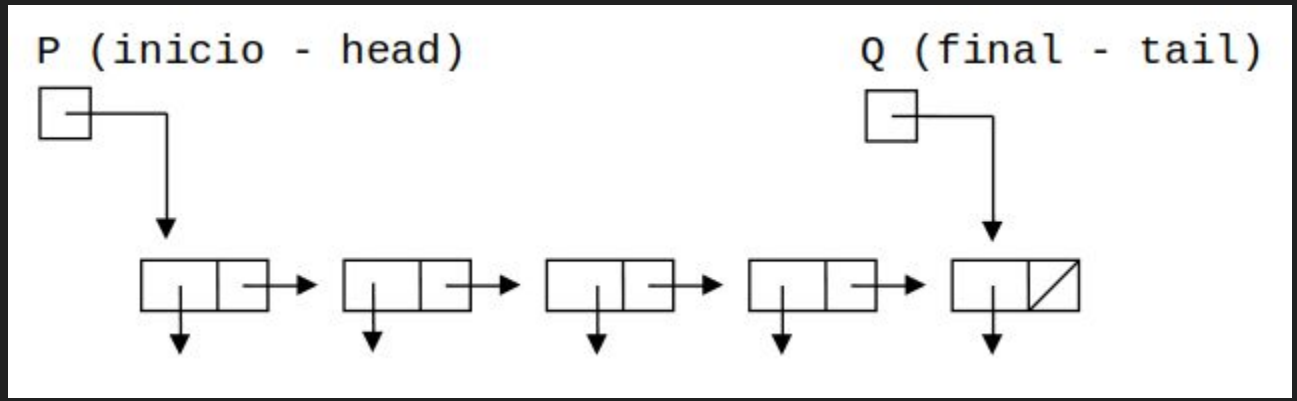


Lista Encadeada (ou Ligada)



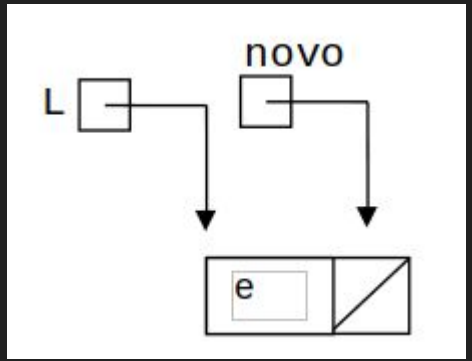
Lista Encadeada

- A informação do item pode ser representada por qualquer tipo ou conjunto de tipos de elementos.
- O ponteiro para o próximo item é um ponteiro para um item do **mesmo tipo**.
- No Python, o campo de informação de cada nó contém na verdade um indicador ou ponteiro para o objeto. Por simplificação podemos supor que a informação está mesmo presente neste campo.



Operações básicas com Lista Encadeada (metalinguagem)

```
# criar uma lista com um primeiro elemento com valor e
novo = Node(e)
novo.prox = None
L.inicio = novo
L.tamanho = 1
```



Operações básicas com Lista Encadeada (metalinguagem)

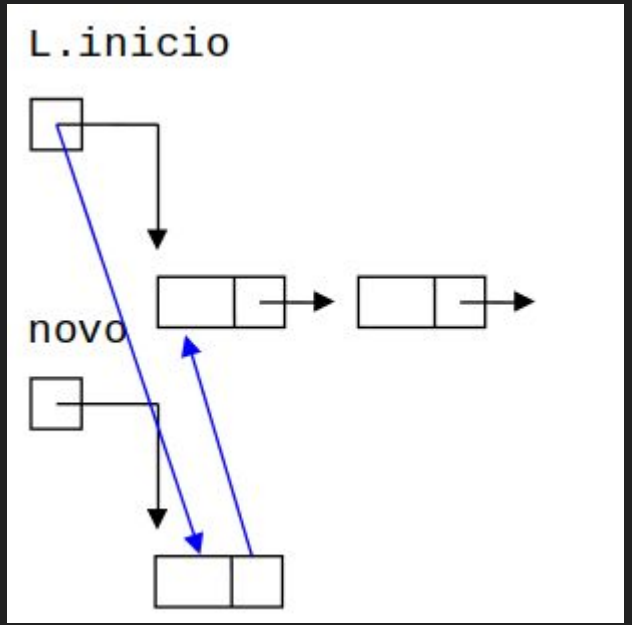
```
# inserir elemento no inicio de lista L

novo = Node(e)

novo.prox = L.inicio

L.inicio = novo

L.tamanho = L.tamanho + 1
```



Operações básicas com Lista Encadeada (metalinguagem)

inserir elemento no final da lista

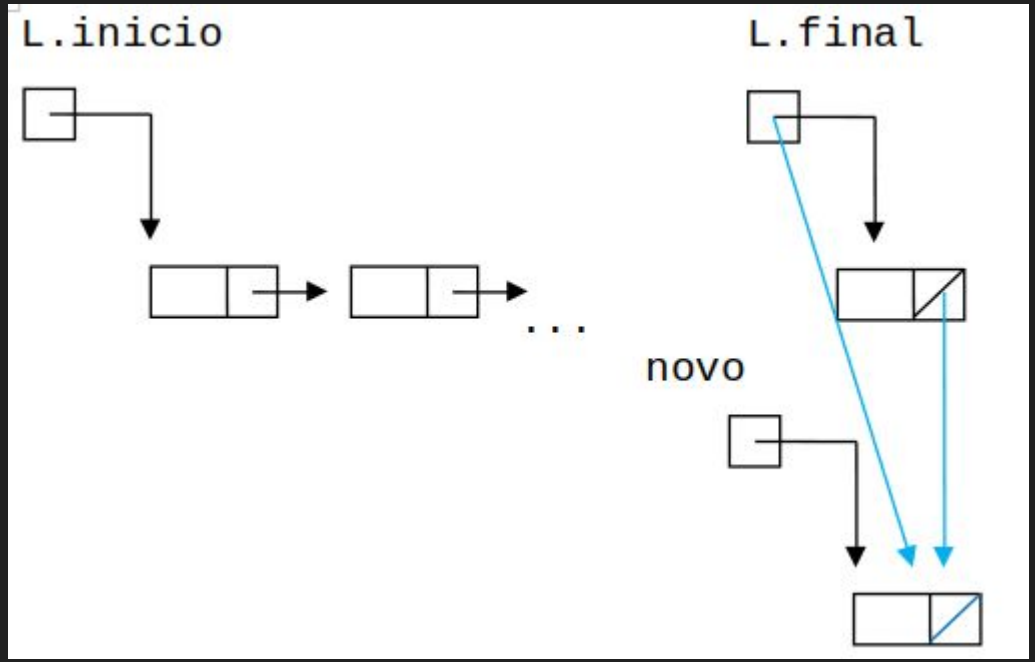
novo = Node(e)

novo.prox = None

L.final.prox = novo

L.final = novo

L.tamanho = L.tamanho + 1



Operações básicas com Lista Encadeada (metalinguagem)

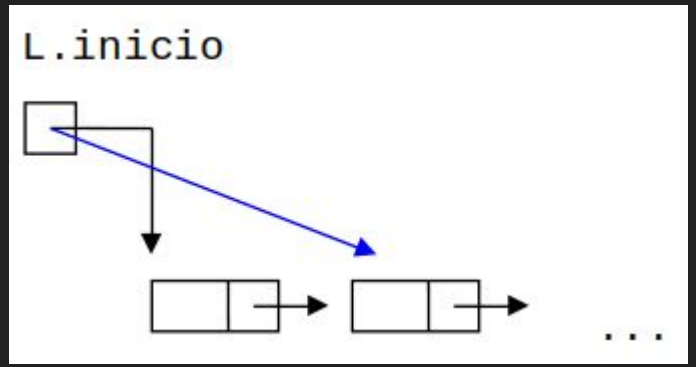
```
# remover elemento do início da lista L

if L.inicio is None:

        # erro – lista vazia

L.inicio = L.inicio.prox

L.tamanho – L.tamanho - 1
```



- (Para remover elemento do final da lista a tarefa não é tão fácil. Precisamos saber quem é o penúltimo. Isso só pode ser feito percorrendo a lista desde o início.)
- (O mesmo ocorre para remover um elemento do meio da lista encadeada. É necessário saber quem é o anterior. *A solução para esse problema é:
  - um nó auxiliar ao percorrer a lista
  - a lista duplamente encadeada

A Classe Lista Encadeada

- A classe de lista será dependente da classe _Node:
  - um elemento ou nó da lista encadeada.
  - é uma classe interna porque não precisa ser conhecida ou referenciada externamente à classe.

```
class ListaEncadeadaSimples :


     class _Node:
        # Classe interna para armazenar um nó
        def __init__ (self, info, prox):
            # iniciar os campos de um nó
            self._info = info  # contéudo ou informação
            self._prox = prox  # referência ao próximo
```

A Classe Lista Encadeada

- Os métodos da classe são:
  - Construtor da classe: __init__()
  - Devolve o tamanho da LE: __len__()
  - Devolve True se LE vazia: is_empty()
  - Devolve o info do primeiro elemento sem removê-lo: first()
  - Adiciona elemento no início: adiciona()
  - Busca elemento (info): busca()
  - Busca elemento (info) devolvendo também o anterior: busca_ant ()
  - Adiciona elemento a frente: adiciona_afrente()
  - Remove elemento a frente: remove_afrente()
  - Criar uma LE a partir de uma lista: CriaLE()
  - Imprimir info da LE: ImprimaLE()

A Classe Lista Encadeada

```
class ListaEncadeadaSimples:
    …
    def __init__(self):
        # cria uma lista vazia
        self._inicio = None  # vazia
        self._tamanho = 0

    def __len__(self):
        # retorna o tamanho da LE
        return self._tamanho
```

Implementação de Lista Encadeada

```
    def is_empty(self):
        # retorna True se LE vazia
        return self._tamanho == 0

    def first(self):
        # retorna o campo info da LE sem remover
        # sinaliza exceção se LE vazia
        if self.is_empty():
            raise Empty("Lista encadeada Vazia")
        return self._inicio._info # elemento inicial da LE

    def adiciona(self, e):
        # adiciona elemento no inicio da LE
        # novo nó referencia o inicio da LE
        novo = self._Node(e, self._inicio)
        # novo nó será o inicio da LE
        self._inicio = novo
        self._tamanho += 1
```

Implementação de Lista Encadeada

```
    def adiciona_afrente(self, p, e):
        # adiciona elemento a frente de p
        # novo nó referencia o mesmo que p
        novo = self._Node(e, p._prox)
        # p referencia o novo nó
        p._prox = novo
        self._tamanho += 1

    def remove_afrente(self, p):
        # remove elemento a frente de p
        # caso particular - p não pode ser o último
        if p._prox is None:
            raise Empty("Nó inexistente")
        # p referencia o mesmo que o próximo
        p._prox = p._prox._prox
        self._tamanho -= 1
```

Implementação de Lista Encadeada

```
def busca(self, e):
    # procura elemento com info = e
    # devolve referência para esse elemento ou None
    # p percorre a lista
    p = self._inicio
    while p is not None:
        if p._info == e: return p # achou
        p = p._prox # vai para o próximo
         # se chegou aqui é porque não achou
     return None
def busca_ant(self, e):
    # procura elemento com info = e
    # devolve uma dupla de referências (p_ant, p)
    # p percorre a lista e pant referencia o anterior
    pant, p = None, self._inicio
    while p is not None:
        if p._info == e:
            return pant, p
        pant, p = p, p._prox
    return pant, p
```

Implementação de Lista Encadeada

```
def CriaLE(self, vet):
    # cria uma LE com elementos do vetor vet
    for k in range(len(vet) - 1, -1, -1):
        self.adiciona(vet[k])

def ImprimeLE(self):
    # só para teste
    p = self._inicio
    k = 1
    print("Imprimindo a lista encadeada:")
    while p is not None:
        print(k, " - ", p._info)
        k = k + 1
        p = p._prox
```

Implementação de Lista Encadeada

```
# Cria lista encadeada
cores = ["vermelho", "preto", "azul", "amarelo", "verde", "branco"]
LEC = ListaEncadeadaSimples()
LEC.CriaLE(cores)
LEC.ImprimeLE()

# Verifica se algumas cores estão na lista
while True:
   cor = input("\nEntre com o nome da cor:")
   if cor == 'fim': break
   if LEC.busca(cor) is None:
       print("A cor", cor, "não está na lista encadeada")
   else:
       print("Encontrada a cor", cor, "na lista encadeada")

print("\nAdicionando laranja e violeta depois e antes da azul")
```

Implementação de Lista Encadeada

```
# Adicionar cor laranja a frente do azul
anterior, atual = LEC.busca_ant("azul")
if atual is None:
   print("Cor azul não está na lista")
else:
   LEC.adiciona_afrente(atual, "laranja")

# Adicionar cor violeta antes da azul
if anterior is None:
   print("Cor azul não tem anterior")
else:
   LEC.adiciona_afrente(anterior, "violeta")

LEC.ImprimeLE()
```

Implementação de Lista Encadeada

```
print("\nRemovendo as cores amarelo e lilás da lista")

# Remove cor amarelo
anterior, atual = LEC.busca_ant("amarelo")
if atual is None or anterior is None:
   print("Cor amarelo não está na lista ou não tem anterior")
else:
   LEC.remove_afrente(anterior)

# Remove cor lilas
anterior, atual = LEC.busca_ant("lilas")
if atual is None or anterior is None:
   print("Cor lilas não está na lista ou não tem anterior")
else:
   LEC.remove_afrente(anterior)

LEC.ImprimeLE()
```

Implementação de Lista Encadeada

- Implementação completa.

Exercícios

Escreva as funções abaixo dentro da classe ListaEncadeadaSimples. Sempre verifique se a função funciona para os casos particulares: lista vazia, lista com um só elemento, primeiro elemento da lista, último elemento da lista, etc.

1. Função Procura(x) que procura nó com info = x e devolve referência a este nó ou None (é a própria função busca (self, e) acima).
2. Supondo que os nós estão em ordem crescente pelo campo info, faça uma função Adiciona_emOrdem(x) que adiciona um novo nó com info igual a x, mantendo a ordem. Para isso é necessário procurar o primeiro nó com info maior que x, mas guardar a referência ao nó anterior para que possa inserir o novo nó à frente dele.
3. Função Conta(x) que devolve a quantidade de nós com info = x.
4. Função ComparaLE(LEX) que compara o conteúdo (campo info) de uma lista encadeada com a lista encadeada LEX. Neste caso, para se comparar a lista encadeada LX com a LEX o comando será LX.Compara(LEX). Outra forma seria escrever a função com 2 parâmetros: Compara(LX, LEX). Faça das duas formas. As funções devem retornar se as listas são iguais ou diferentes.

A subclasse _Node

- Na classe ListaEncadeadaSimples a classe _Node é definida como uma classe interna: evitar que os atributos _info e _prox sejam conhecidos e usados pelas funções que usam esse objeto.
- Usando a classe _Node no mesmo nível da classe ListaEncadeadaSimples, a única modificação a ser feita seria no comando que cria um novo objeto _Node:

  novo = _Node(e, p._prox) e novo = _Node(e, self._inicio)

  ao invés de:

  novo = self._Node(e, p._prox) e novo = self._Node(e, self._inicio)

Recursão em Listas Encadeadas

- Listas encadeadas simples são claramente recursivas.
  - Assim, admitem também algoritmos recursivos.
  - Cada nó de uma lista encadeada pode ser entendido como um ponteiro para uma nova lista encadeada.

Lista encadeada =

{

Vazia ou

Elemento contendo um campo de informação e uma referência para uma Lista Encadeada

}

Recursão em Listas Encadeadas

- Listas encadeadas simples são claramente recursivas.
  - Assim, admitem também algoritmos recursivos.
  - Cada nó de uma lista encadeada pode ser entendido como um ponteiro para uma nova lista encadeada.

Lista encadeada =

{

Vazia ou

Elemento contendo um campo de informação e uma referência para uma Lista Encadeada

}

Claramente a natureza de definição de listas nos permite iterar sobre seus nós utilizando recursão!

Assim, uma lista encadeada pode sempre ser um par (info, prox), com prox como uma referência para uma nova lista encadeada.

Recursão em Listas Encadeadas

```
class OutraListaEncadeadaSimples :
    ''' Nesta nova definição de LE, Uma LE é sempre um par:
        (info, prox), onde prox é uma lista encadeada.
        Quando info == None a lista encadeada é vazia.
    '''

    def __init__ (self, info = None, prox= None):
        # cria nova LE
        self._info = info
        self._prox = prox

    def is_empty(self):
        # retorna True se LE vazia
        return self._info == None

    def info(self):
        # retorna o campo info da LE sem remover.
        # sinaliza exceção se LE vazia
        if self.is_empty():
            raise Empty("Lista encadeada Vazia")
        return self._info # informação desta LE
```

Recursão em Listas Encadeadas

```
def adicionaLE(self, e):
    # adiciona uma LE com info = e antes desta LE
    # novo nó referencia o inicio da LE
    novaLE = OutraListaEncadeadaSimples(e, self)
    return novaLE

def busca(self, e):
    # procura LE com info = e. Devolve referência
    # para a LE encontrada ou None
    # A função é recursiva
    # p percorre a lista
    if self.is_empty(): return None
    if self._info == e: return self
    return self._prox.busca(e)
```

Recursão em Listas Encadeadas

```
def CriaLE(self, lista):
    # cria uma LE com elementos contidos em lista
    for k in range(len(lista) - 1, -1, -1):
        self.adiciona(lista[k])
 def ImprimeLE(self):
    # imprime toda a LE
    p = self
    k = 1
    print("Imprimindo a lista encadeada:" )
    while p is not None:
        print(k, " - ", p._info)
        k = k + 1
        p = p._prox
```

Recursão em Listas Encadeadas

```
# Testes
# Cria lista encadeada
cores = ["vermelho", "preto", "azul", "amarelo", "verde", "branco"]
LEC = OutraListaEncadeadaSimples()
for cor in cores:
   LEC = LEC.adicionaLE(cor)
LEC.ImprimeLE()

# Procura alguns elementos
outrascores = ["laranja", "violeta", "lilas"]
for cor in cores + outrascores:
   if LEC.busca(cor) != None: print(cor, "está na LE")
   else: print(cor, "não está na LE")
```

- Código completo.

Exercícios

5. Refaça os métodos CriaLE e ImprimeLE acima de maneira recursiva.
6. Com a mesma definição acima, refaça o exercício 3 de forma recursiva.

Alocação Dinâmica - Pilhas e Filas

- Pode-se implementar Pilhas e Filas como listas encadeadas.
- Cada estrutura tem as suas particularidades.
  - Classes construídas separadamente: ambas utilizam a mesma classe _Node já definida anteriormente.

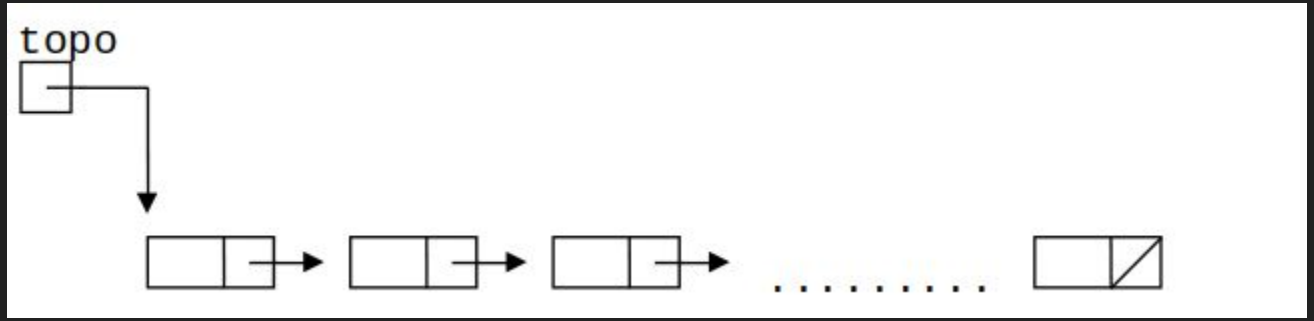
```
class _Node:

   # classe interna para armazenar um nó

   def __init__ (self, info, prox):
       # iniciar os campos de um nó
       self._info = info
       self._prox = prox
```

Pilhas com listas encadeadas (LIFO)

- Pilha: operações realizadas com topo
  - Naturalmente, o topo da pilha é definido como o início da lista encadeada.



Pilhas com listas encadeadas (LIFO)

- Funções da classe PilhaListaEncadeada:
  - Empilhar elemento x – push(x)
  - Desempilhar elemento – pop()*

- Outras operações auxiliares:
  - Iniciar uma pilha – init()
  - Consultar o valor do elemento do topo – top()*
  - Retornar True se a pilha está vazia – is_empty()
  - Retornar o tamanho atual da pilha – len()
- ***Devem informar se a pilha está vazia.***

Pilhas com listas encadeadas (LIFO)

```
class Empty(Exception):
   pass
class PilhaListaEncadeada:
   # implementa uma pilha usando uma LE simples
   # classe _Node - interna
   class _Node:

       __slots__ = '_info', '_prox'

       def __init__ (self, info, prox):
           # inicia os campos
           self._info = info
           self._prox = prox
```

Pilhas com listas encadeadas (LIFO)

```
    # métodos de pilha
    def __init__ (self):
        # cria uma pilha vazia
        self._topo = None # vazia
        self._tamanho = 0 # tamanho da pilha

    def __len__(self):
        # retorna o tamanho da pilha
        return self._tamanho

    def is_empty(self):
        # retorna True se pilha vazia
        return self._tamanho == 0

    def push(self, e):
        # adiciona elemento ao topo da pilha
        novo = self._Node(e, self._topo)
        self._topo = novo
        self._tamanho += 1
```

Pilhas com listas encadeadas (LIFO)

```
def top(self):
    # retorna sem remover o topo da pilha
    # sinaliza exceção se pilha vazia
    if self.is_empty():
        raise Empty("Pilha Vazia!")
    return self._topo._info # topo da pilha
def pop(self):
    # remove e retorna o topo da pilha
    # sinaliza exceção se pilha vazia
    if self.is_empty():
        raise Empty("Pilha Vazia!")
    val_topo = self._topo._info
    self._topo = self._topo._prox # pula o primeiro elemento
    self._tamanho -= 1
    return val_topo
def ImprimeLE(self):
    p = self._topo
    print("Imprimindo a pilha!")
    while p is not None:
        print(p._info)
        p = p._prox
```

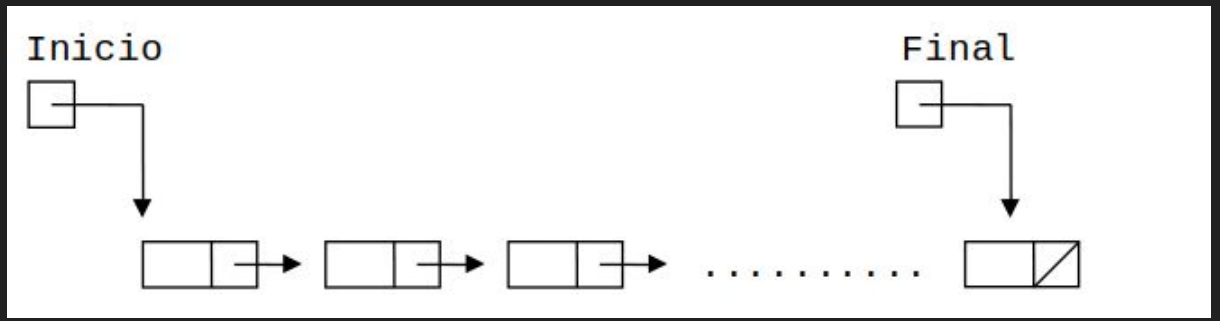
Pilhas com listas encadeadas (LIFO)

```
# Testes
# Cria uma pilha
P = PilhaListaEncadeada()
# adiciona 10 elementos
for k in range(10): P.push(k)
P.ImprimeLE()
# remove 5 elementos
for k in range(5): P.pop()
P.ImprimeLE()
# algumas informações da pilha
print("\ntopo da pilha = ", P.top())
print("tamanho da pilha = ", len(P))
# remove 6 elementos - vai dar excessão
for k in range(6): P.pop()
```

- Código Completo.

Filas com Listas Encadeadas (FIFO)

- Ponteiros para início e fim da fila: operações de inserir e remover elementos
- Abordagem intuitiva: início e fim da fila coincidem com o início e fim da lista encadeada.



Filas com Listas Encadeadas (FIFO)

- Operações:
  - Adicionar um elemento x – enqueue(x)
  - Retirar um elemento – dequeue() - deve avisar se a fila está vazia
  - Consultar o valor do primeiro elemento – first() - deve avisar se a fila está vazia
  - Retornar True se a fila está vazia – is_empty()
  - Retornar o tamanho atual da fila – len()

Implementação de Filas com Listas Encadeadas

```
class Empty(Exception):
   pass
class FilaListaEncadeada:
   # implementa uma fila usando uma LE simples
   # classe _Node - interna
   class _Node:
       __slots__ = '_info', '_prox'
       def __init__ (self, info, prox):
           # inicia os campos
           self._info = info
           self._prox = prox
   # métodos de fila
   def __init__ (self):
       # cria uma fila vazia
       self._inicio = None # vazia
       self._final = None  # vazia
       self._tamanho = 0   # tamanho da pilha
```

Implementação de Filas com Listas Encadeadas

```
    def __len__(self):
        # retorna o tamanho da fila
        return self._tamanho
    def is_empty(self):
        # retorna True se fila vazia
        return self._tamanho == 0
    def first(self):
        # retorna sem remover o inicio da fila
        # sinaliza exceção se fila vazia
        if self.is_empty():
            raise Empty("Fila Vazia!")
        return self._inicio._info  # elemento inicial da fila
    def enqueue(self, e):
        # adiciona elemento ao final da fila
        novo = self._Node(e, None) # novo nó será o fim da fila
        # caso especial - fila vazia
        if self.is_empty():
            self._inicio = novo
        else:
            self._final._prox = novo
        # atualiza o fim da fila e o tamanho
        self._final = novo
        self._tamanho += 1
```

Implementação de Filas com Listas Encadeadas

```
def dequeue(self):
    # remove e retorna o elmento do inicio da fila
    # sinaliza exceção se fila vazia
    if self.is_empty():
        raise Empty("Fila Vazia")
    val = self._inicio._info
   # pula o primeiro elemento
    self._inicio = self._inicio._prox
    self._tamanho -= 1
    # caso especial - a fila ficou vazia
    if self.is_empty():
        self._final = None
    return val
def ImprimeLE(self):
    ''' só para teste.'''
    p = self._inicio
    print("Imprimindo a fila  - lista encadeada:)"
    k = 1
    while p is not None:
        print(k, " - ", p._info)
        p = p._prox
        k += 1
```

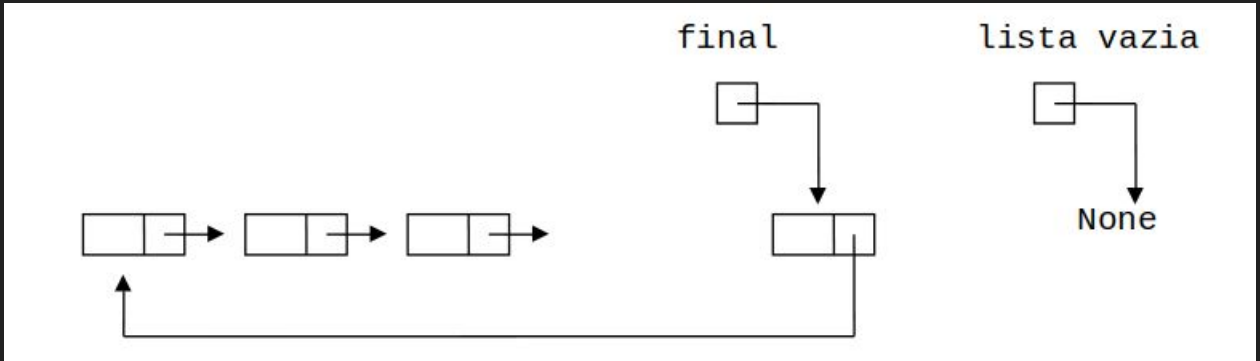
Implementação de Filas com Listas Encadeadas

```
# Testes


# Cria uma fila em lista encadeada
F = FilaListaEncadeada()
# adiciona 10 elementos
for k in range(10): F.enqueue(k)
F.ImprimeLE()
# remove 6 elementos
for k in range(6): F.dequeue()
F.ImprimeLE()
# algumas informações da fila
print("\nprimeiro elemento da fila = ", F.first())
print("tamanho da fila = ", len(F))
# remove 6 elementos - vai dar exceção
for k in range(6): F.dequeue()
```

Lista Circular

- O último item aponta para o primeiro.
- Pode-se acessar todos os elementos a partir de qualquer elemento:
  - Conveniente manter referências para o início e fim da lista encadeada circular.
  - **Se mantivermos a referência apenas para o fim, temos acesso imediato ao início: basta uma referência, pois com ela podemos percorrer toda a lista.**



Lista Circular

- Interesse de aplicação da lista circular: temos certo número de elementos aos quais devemos oferecer algum serviço ou efetuar alguma ação de maneira contínua respeitando a ordem.
  - Exemplo: sistema operacional que tem que atender aos vários processos em execução num sistema de tempo compartilhado (time-sharing), alocando a CPU a cada um deles por um período.
- Operações:
  - Inserção de elemento no final da lista: colocá-lo à frente do elemento final, tornando este o novo final.
  - Remoção de elemento do início da lista: retirar o elemento que está à frente do elemento final.
    - Para o caso em que o primeiro elemento é usado e em seguida volta para o final da lista: adiantar de um nó a referência do final - o que era o início passa a ser o final.

Implementação de Lista Circular

- Para a classe ListaCircularListaEncadeada, usamos as mesmas funções anteriores com a adição da rotate() que adianta a referência de final da lista.

```
class ListaCircularListaEncadeada:

    # implementa uma lista circular com LE simples
    # classe _Node - interna
    class _Node:

        __slots__ = '_info', '_prox'

        def __init__ (self, info, prox):
            # inicia os campos
            self._info = info
            self._prox = prox
```

Implementação de Lista Circular

```
# métodos de lista circular
def __init__ (self):
    # cria uma lista circular vazia
    self._final = None # vazia
    self._tamanho = 0  # tamanho da lista

def __len__(self):
    # retorna o tamanho da lista
    return self._tamanho

def is_empty(self):
    # retorna True se lista vazia
    return self._tamanho == 0

def first(self):
    # retorna o valor sem remover o início da lista
    # sinaliza exceção se lista vazia
    if self.is_empty():
        raise Empty("Lista Circular Vazia!")
    primeiro = self._final._prox
    return primeiro._info
```

Implementação de Lista Circular

```
def enqueue(self, e):
    # adiciona elemento no final da lista
    novo = self._Node(e, None)
    if self.is_empty():
        novo._prox = novo    # circular
    else:
        novo._prox = self._final._prox # aponta ao primeiro
        self._final._prox = novo       # aponta ao novo
    self._final = novo
    self._tamanho += 1
def dequeue(self):
    # remove e retorna o primeiro elemento da lista
    # sinaliza exceção se lista vazia
    if self.is_empty():
        raise Empty("Fila Circular Vazia!")
    atual = self._final._prox
    # caso especial - 1 só elemento
    if self._tamanho ==1:
        self._final = None
    else:
        self._final._prox = atual._prox # pula o primeiro
    self._tamanho -= 1
    return atual._info
```

Implementação de Lista Circular

```
def rotate(self):
    # avança a referência de final de 1 nó
    if self._tamanho > 0:
        self._final = self._final._prox

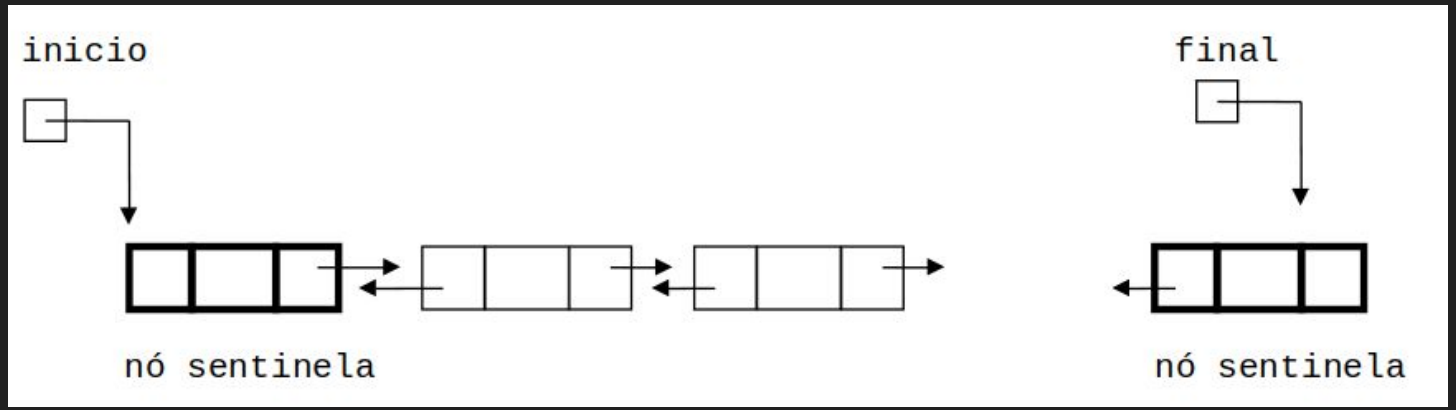
def ImprimeLE(self):
    if self._tamanho == 0: return
    p = self._final._prox
    print("Imprimindo a lista")
    while True:
        print(p._info)
         if p is self._final: return
                    p = p._prox
```

Implementação de Lista Circular

```
# Testes
# Cria uma fila circular em lista encadeada
LC = ListaCircularListaEncadeada()
# adiciona 10 elementos
for k in range(10): LC.enqueue(k)
LC.ImprimeLE()
# remove 5 elementos
for k in range(5): LC.dequeue()
LC.ImprimeLE()
# algumas informações da lista circular
print("\núltimo elemento = ", LC.first())
print("tamanho da lista = ", len(LC))
# remove 6 elementos - vai dar excessão
for k in range(6): LC.dequeue()
```

Listas Duplamente Encadeadas

- Problemas da lista encadeada simples:
  - Adicionar ou remover elementos no meio ou no final - necessário saber o elemento anterior.
  - A lista pode ser percorrida numa só direção.
- Solução: lista duplamente encadeada.
  - Cada elemento tem duas referências: para o nó seguinte e para o anterior
  - Também é interessante manter um ponteiro para o elemento final.



Listas Duplamente Encadeadas

- Para facilitar: o primeiro e o último elementos são “sentinelas” da lista e não tem a função de nó.
  - São usados apenas para facilitar a navegação: seus campos info são None para diferenciá-los dos reais nós da lista.

Implementação de Listas Duplamente Encadeadas

- Para facilitar: o primeiro e o último elementos são “sentinelas” da lista e não tem a função de nó.

```
class ListaDuplamenteencadeada:

   # operações sobre uma lista duplamente encadeada

   # classe _Node - interna
   class _Node:

       __slots__ = '_info', '_prev', '_prox'

       def __init__ (self, info, prev, prox):
           # inicia os campos
           self._info = info
           self._prev = prev
           self._prox = prox
```

Implementação de Listas Duplamente Encadeadas

```
# métodos de lista duplamente encadeada
def __init__ (self):
    # cria uma lista DE vazia
    self._inicio = self._Node(None, None, None) # vazia
    self._final = self._Node(None, None, None) # vazia
    self._inicio._prox = self._final
    self._final._prev = self._inicio
    self._tamanho = 0  # tamanho da lista

def __len__(self):
    # retorna o tamanho da lista
    return self._tamanho

def is_empty(self):
    # retorna True se lista vazia
    return self._tamanho == 0
```

Implementação de Listas Duplamente Encadeadas

```
    def adicionar_entre(self, e, anterior, sucessor):
        # adiciona elemento entre 2 outros
        # retorna o novo nó
        novo = self._Node(e, anterior, sucessor)
        anterior._prox = novo
        sucessor._prev = novo
        self._tamanho += 1
        return novo

    def remove(self, node):
        # remove nó da lista e retorna seu valor
        anterior = node._prev
        sucessor = node._prox
        anterior._prox = sucessor
        sucessor._prev = anterior
        self._tamanho -= 1
```

Listas Encadeadas vs. Listas Sequenciais

- Lista encadeada: manipulação + complexa.
  - Porém, a vantagem está na inserção e remoção de elementos.
    - Operações de inserção e remoção independem da eficiência dos métodos internos de alocação de memória (no caso do Python: append(), pop(), etc.).
    - A alocação de memória, quando necessária, está por conta do seu programa.
- Lista sequencial: manipulação + simples.
  - Porém, depende da eficiência de implementação dos métodos internos de alocação de memória.



Obrigado!

Contato:

- la.souza@inf.ufes.br
- github.com/luisouza
- https://scholar.google.com.br/citations?user=3gMIEPYAAAAJ&hl=pt-BR
- https://www.researchgate.net/profile/Luis-Souza-Jr